

Due: Monday, March 9th at 11:59 pm

- This homework will cover the Fourier analysis, filtering, Bode plots, frequency response, and dynamic responses of systems.
- In all of the questions, **show your work**, not just the final answer. Unless we explicitly state otherwise, you may expect full credit only if you explain your work succinctly, but clearly and convincingly. For coding questions, attach a screenshot of your code and output.
- Present your answers with a **suitable number of significant figures** for each question. Show your work, including a mathematical formula or the MATLAB or Python code you wrote, before reaching the result. You may need to install the Signal Processing Toolbox if using MATLAB.
- Throughout this assignment, neglect systematic (bias) errors. Also, assume a normal distribution for the underlying distribution (population) if necessary.
- If you have a confirmed disability that precludes you from complying fully with these instructions or with any other parameter associated with this problem set, please alert us immediately about reasonable accommodations afforded to you by the DSP Office on campus.
- **Start early. Some of the material is prerequisite material not covered in lecture; you are responsible for finding resources to understand it.**

Deliverables

Submit a PDF of your homework to the **Gradescope assignment** entitled “{Your Name} HW3”. **You must typeset your homework in L^AT_EX (submit PDF format, not .doc/.docx format)**. Mac Preview, PDF Expert, and FoxIt PDF Reader, among others, have tools to let you sign a PDF file. We want to make *extra clear* the consequences of cheating.

0 Honor Code

I will adhere to the Berkeley Honor Code: specifically, as a member of the UC Berkeley community, I act with honesty, integrity, and respect for others. Failure to comply with these guidelines can be considered an academic integrity violation. Please email Professor Anwar ganwar@berkeley.edu if you have any questions!

- **List all collaborators. If you worked alone, then you must explicitly state so. Read the following statement and sign below if you agree:**

“I certify that all solutions in this document are entirely my own and that I have not looked at anyone else’s solution. I have given credit to all external sources I consulted.”

Signature : _____ Date : _____

While discussions are encouraged, *everything* in your solution must be your (and only your) creation. Furthermore, all external material (i.e., *anything* outside lectures and assigned readings, including figures and pictures) should be cited properly. We wish to remind you that consequences of academic misconduct are *particularly severe*!

- **Violation of the Code of Conduct will result in a **zero** on this assignment and may also result in disciplinary action.**

1 Fourier meets Coding [28 pts]

The purpose of this assignment is to familiarize you with concepts related to Fourier analysis, sometimes referred to as spectral analysis. Fourier analysis has wide applications from electrical systems and vibratory systems to signal processing and even fluid mechanics.

The Continuous Time Fourier Transform (CTFT) Synthesis Equation is defined as follows:

$$X(\omega) = \int_{-\infty}^{\infty} x(t)e^{-i\omega t} dt \quad (1)$$

Here, again, ω is in units of rad/s, not Hz. Note how similar this definition is to the definition of the Laplace Transform, which you may be more familiar with. It is important to recognize that no information is gained or lost when evaluating the Fourier transform. One can transform back and forth between the time and frequency domains without losing any information. Similarly, no information is created. These properties are retained when the Fourier transform is discretized.

The Discrete Fourier Transform (DFT) is analogous to the continuous formulation:

$$F_k = \sum_{m=1}^N f_m \cdot e^{-\frac{i2\pi}{N}(k-1)(m-1)}, \quad \forall k \in \mathbb{Z} \quad (2)$$

where N is the total number of discrete points (e.g. data points) in one period T , f_m is the real value of the m^{th} data point, and F_k is k^{th} complex data point of the transformed function. Time corresponds to m ; frequency corresponds to k . It should be noted, however, that m and k are simply labels for each data point, not the actual value of time and frequency. For example, the m^{th} data point f_m corresponds to the time $t = \frac{(m-1)T}{N}$. Similarly, the k^{th} transformed data point F_k corresponds to the frequency (in rad/s) $\omega = \frac{(k-1)\omega_0}{N}$.

In practice, the DFT equation above is rarely used. It is cumbersome and calculating every term in the series is unnecessary. Instead, the Fast Fourier Transform (FFT) is widely used. The FFT is a more computationally efficient method for calculating Eq. (2). The FFT is a method to calculate the DFT.

- (a) (i) [6 pts] Write and show your **own** MATLAB code that will evaluate $(F_1, \dots, F_N)$ directly using Eq. (2) for a **unitless** dataset in **xt1** (in **HW3data.mat**)

Solution:

```
1 x = xt1;
2 N = length(x); % size of the sample
3
4 for k=1:N
5     A(k)=0;
6     for m=1:N
7         A(k)=A(k)+x(m)*exp((-1i)*2*pi*(m-1)*(k-1)/N);
8     end
9 end
10 toc; % execution time measurement ends
11
12 tic; % execution time measurement begins
13 Y = fft(xt1);
14 toc; % execution time measurement ends
```

- (ii) [4 pts] Compare the time it takes for your code to run compared to that for the **fft** function in MATLAB. *Hint:* use the **tic** and **toc** functions to measure code execution time.

Solution: Our code took approximately 4 seconds to terminate on an ordinary computer while the build-in MATLAB function **fft** needed only 0.01 seconds. The main cost comes from the N^2 terms computed in the double for loop which is sub-optimal from a computational perspective.

- (b) (i) [4 pts] Given $(f_1, \dots, f_N) \in \mathbb{R}^N$, assume you now evaluate $(F_1, \dots, F_N)$ by a **slow calculator** that computes one natural exponent e^z or one basic operation (i.e. addition and multiplication) a second. How much time does it take to evaluate $(F_1, \dots, F_N)$ when naively using Eq. (2)? Ignore data input/output time.

Solution: It should take approximately (1 flip-sign operation + 3 multiply operations + 2 minus operations + 2 multiply operations + 1 exponent + 1 multiply operation) * (N sum operations) * (N calculations for each F_k).

- (ii) [3 pts] **The first step from DFT to FFT:** Discuss how to reduce the amount of computation time of the naive DFT, regardless of what real values are assigned to $(f_1, \dots, f_N)$. *Hint:* express Eq.(2) in matrix form and find symmetries.

Solution: We should focus on “symmetries” occurring during the computation. It can be found that there are two major symmetries in the DFT formula.

First, the computation amount can be reduced by **recycling the exponent calculations when the values assigned for m and k are simply switched**. For instance, when we calculate $e^{\frac{-i2\pi}{N}(k-1)(m-1)}$, the value for $k-1=3, m-1=2$ is the same as the value for $k-1=2, m-1=3$. It enables us to avoid unnecessary exponent calculations.

Secondly, we can take advantage of another symmetry. Because

$$\begin{aligned} F_{N-k+2} &= \sum_{m=1}^N f_m e^{\frac{-i2\pi}{N}(N-k+1)(m-1)} \\ &= \sum_{m=1}^N f_m e^{-i2\pi} e^{\frac{i2\pi}{N}(k-1)(m-1)} = \sum_{m=1}^N f_m e^{\frac{-i2\pi}{N}(k-1)(m-1)} \\ &= \sum_{m=1}^N f_m e^{\frac{-i2\pi}{N}(k-1)(m-1)} = \overline{F_k}, \end{aligned}$$

for all k . In the second step we used the fact that $e^{-i2\pi} = 1$. Thus, we are able to **compute only the first half of the transformed data points and then take the complex conjugates for the rest data points**. For example, once F_2 is computed as $a + bi$, F_N is automatically determined as its complex conjugate $a - bi$, without the need to sum up N terms.

Much like the continuous Fourier transform, the DFT assumes that the function is *periodic*, which may not be the case. Furthermore, the DFT violates one of the original assumptions: that the function is finitely discontinuous. By definition, a periodic discrete function is *not* finitely discontinuous. Violating these assumptions leads to some interesting behavior. For the sake of clarity, the remainder of this document will discuss Fourier analysis in the context of data collection, unless explicitly stated otherwise.

The assumption of periodicity in the time domain requires that the transformed function F_k is also periodic in the frequency domain. As a result, F_k reflects at a certain frequency called the Nyquist frequency. This reflection is the cause of *aliasing*. Aliasing is when a high frequency signal appears to the measurer as a lower frequency signal. The Nyquist frequency occurs at half of the sampling frequency, i.e.,

$$f_{\text{Nyquist}} = \frac{1}{2} f_s = \frac{1}{2} \cdot \frac{N}{T}, \quad (3)$$

where f in Eq.(3) means frequency in Hz, having the relationship with ω as $\omega = 2\pi f$. In other words, any data points beyond the Nyquist frequency are meaningless. They are just a reflection of the data below the Nyquist frequency.

Furthermore, the frequency function can reflect non-positive frequencies like $k = 0, -1, -2, \dots$. However, we only evaluated the DFT for positive integers for k . Why is that? When doing Fourier analysis, we often plot the magnitude of the function at each frequency. The magnitude contribution of a negative frequency is identical to its positive counterpart (recall the relationship between A_{-n} and A_n obtained from (a)). That is, half of the “energy” of the function is in the negative frequency domain. Rather than calculate those magnitudes with Eq.(2), it is easier to double the positive frequency magnitudes to account for the lost “energy.” It has become convention to double the magnitude (except at zero and Nyquist frequencies) when processing DFT results.

- (c) (i) [3 pts] Calculate and numerically show the Nyquist frequency for the dataset in `xt1`. Use the fact that the data point `xt1(k)` is measured at time `t(k)` in seconds for any `k`.

Solution: We obtained

$$f_{\text{sample}} = \frac{1}{\Delta T_{\text{sample}}} = 2600 \text{ samples per second,}$$

where ΔT_{sample} is the time interval between two adjacent sample collections. Hence,

$$f_{\text{Nyq}} = \frac{1}{2} f_{\text{sample}} = 1300 \text{ Hz.}$$

- (ii) [3 pts] Modify the code you wrote for (b). First, truncate your DFT vector so that only frequencies below the Nyquist frequency are included. Second, account for the reflection into the negative frequency domain. Show the code you modified.

Solution: We should truncate the data such that no frequency contents whose frequency is beyond 1300 Hz are included. Also, we should multiply all data points (except for the zero and Nyquist frequency data points) by two in order to account for negative reflection. These will be evaluated in the plots provided in (d).

Please also refer to the MATLAB code below.

```

1 % Nyquist frequency calculation
2 T = t(2) - t(1);           % sampling period
3 Fs = 1/T                   % sampling frequency
4 Fn = Fs/2                  % Nyquist frequency
5
6 % calculation using DFT function above
7 P2 = abs(A/N);             % normalization
8 P1 = P2(1:N/2+1);          % truncation of the DFT vector
9 P1(2:end-1) = 2*P1(2:end-1); % doubling the magnitudes

```

After (c), your code should look similar to the code in MATLAB's `fft()` documentation [here](#) (also, see Figure 1). However, the first line in Figure 1 remains unexplained. The first line is a normalization of the data by the number of data points in the set (`L`, in Figure 1). Since the DFT is a summation, more data points generally equate to a larger magnitude. Normalization allows us to compare data sets of different sizes. This form of normalization is one convention; it is not the only form of normalization available.

```

P2 = abs(Y/L);
P1 = P2(1:L/2+1);
P1(2:end-1) = 2*P1(2:end-1);

```

Figure 1: Part of MATLAB's `fft()` documentation.

Furthermore, the first line takes the absolute value of the frequency function (`Y`, in Figure 1). Before applying the absolute value, the frequency function is complex, containing both magnitude and phase/argument information of each frequency content. The absolute value of the frequency function is a power spectrum of the frequency content; it must be real-valued.

Finally, note that the third line in Figure 1 omits the first index of `P1`, which should correspond to $\omega_n = 0$. When $\omega_n = 0$, the only content is the mean value of the periodic function (e.g., the DC offset in electrical signals). Since the $\omega_n = 0$ value is not reflected into the negative frequency domain, it does not need to be doubled, thus it is omitted in line 3. The same argument holds when ω_n corresponds to the Nyquist frequency (i.e., $\omega_n = 2\pi f_{\text{Nyq}}$). At this point, the DFT data should be fully processed for interpretation.

- (d) (i) [3 pts] Plot the single-sided power spectrum from your DFT result. Make sure to have the data points match the correct frequencies (in Hz) and magnitudes (using the normalization in Figure 1).

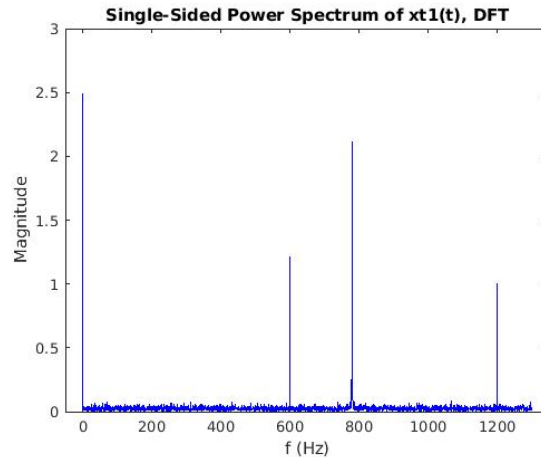
Solution: Please refer to the following code below to plot the power spectrum.

```

1 f = Fs*(0:(N/2))/N;
2 figure(1)
3
4 plot(f,P1,'blue')           % DFT plot
5 title('Single-Sided Power Spectrum of xt1(t), DFT')
6 xlabel('f (Hz)')
7 ylabel('Magnitude')

```

The power spectrum obtained from the DFT computation is depicted below



- (ii) [1 pt] Report the frequencies and magnitudes of all of the notable peaks seen in the power spectrum.

Solution: There are in total 4 outstanding peaks in the spectrum. Their frequencies and magnitudes are specified in the Table below.

Frequency f [Hz]	0	600	780	1200
Magnitude	2.49188	1.20967	2.11589	1.00488

- (e) [1 pt] **Validation:** Conduct the same procedure as Problem 4 but use MATLAB's `fft()` instead of your DFT function. Compare the FFT plot with the DFT one and confirm that they are the same.

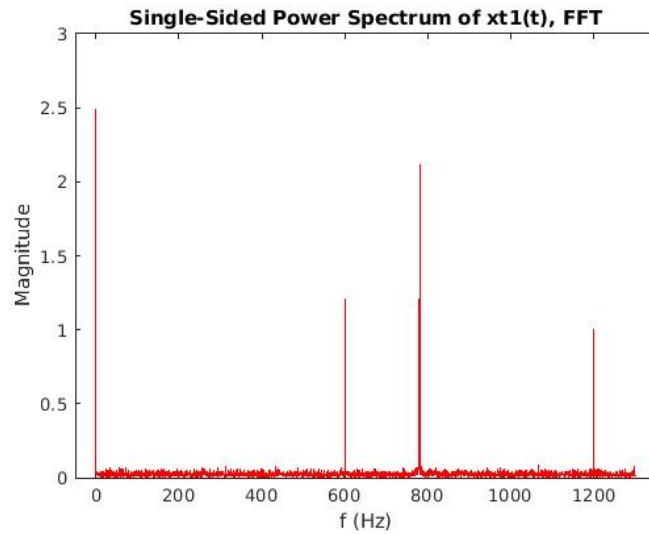
Solution: Please refer to the following code.

```

1 P4 = abs(Y/N);           % Y from fft(xt1);
2 P3 = P4(1:N/2+1);
3 P3(2:end-1) = 2*P3(2:end-1);
4
5 figure(2)
6
7 plot(f,P3,'red')         % FFT plot
8 title('Single-Sided Power Spectrum of xt1(t), FFT')
9 xlabel('f (Hz)')
10 ylabel('Magnitude')
11 xlim([-50 1350]); ylim([0 3]);

```

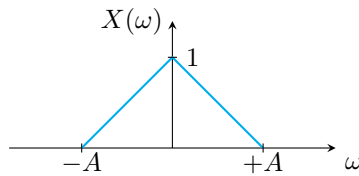
The power spectrum obtained from the FFT computation is depicted in the Figure below. The DFT and FFT plots are identical. The FFT plot also has 4 outstanding peaks with the same magnitudes as the DFT plot as well as end at 1300 Hz.



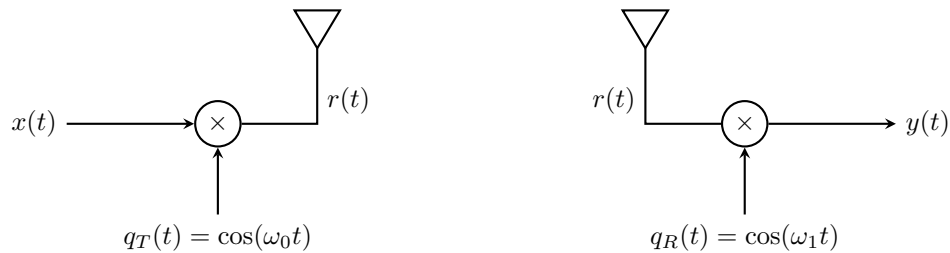
While the results from Problems (d) and (e) are useful, there are several other useful tricks for clarifying the results of DFT or FFT. One of the tricks is windowing. Windows are piecewise functions that are identically zero when outside the data domain but multiply the data by some factor within the data domain. Windows ensure that a set of data meets the requirement of periodicity: the data begins and ends at the same value. When applying a DFT on a finite set of data, a rectangular window is implicitly being applied. A rectangular window multiplies the function by 1 within the domain; multiplies by zero outside the domain. Other common windows include the Hann/Hamming windows and Blackman windows.

2 Heterodyning and Amplitude Modulation [15 pts]

We are given a bandlimited continuous time transmission signal $x(t)$ with the following triangular spectrum shown below.



A spectrum is a graph that shows how much of each frequency is present in the signal. The following diagram shows an (AM) amplitude modulation-demodulation scheme to communicate the signal $x(t)$ to a receiver. *Note:* that the circle with $\times$ means multiplication and the triangle is the antenna that transmits and receives.



There is also a frequency mismatch (ϵ) between the transmitter (LHS) and receiver (RHS) carrier signals q_T and q_R , respectively. In particular, assume that

$$0 < \epsilon \ll A \quad \text{and} \quad A < \omega_0 = \omega_1 + \epsilon$$

- (a) [3 pts] Determine reasonably simple expressions for the signals r and y in terms of ω_0 . You may or may not find the following trigonometric identity useful:

$$\cos \alpha \cos \beta = \frac{1}{2} [\cos(\alpha + \beta) + \cos(\alpha - \beta)]$$

Solution: We can find $r[n]$ through a simple substitution:

$$r(t) = x(t)q_T(t) \implies r(t) = x(t) \cos(\omega_0 t)$$

And proceed similarly to find $y[n]$, making use of the provided trigonometric identity.

$$\begin{aligned} y(t) &= r(t)q_R(t) \\ &= x(t) \cos(\omega_0 t) \cos(\omega_1 t) \\ &= \frac{x(t)}{2} (\cos((\omega_0 + \omega_1)t) + \cos((\omega_0 - \omega_1)t)) \\ &= \frac{x(t)}{2} (\cos((2\omega_0 - \epsilon)t) + \cos(\epsilon t)) \end{aligned}$$

Therefore, the signals r and y in terms of ω_0 are

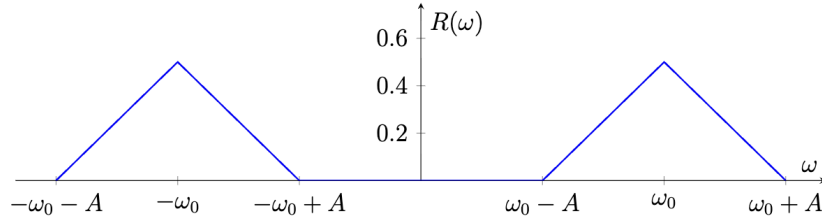
$$r(t) = x(t) \cos(\omega_0 t) \quad \text{and} \quad y(t) = \frac{x(t)}{2} (\cos((2\omega_0 - \epsilon)t) + \cos(\epsilon t))$$

- (b) [10 pts] Provide well-labeled plots of $R(\omega)$ and $Y(\omega)$, the spectra of the signals r and y , respectively.

Solution:

Note the general CTFT pair $e^{j\omega_0 t} \xleftrightarrow{\mathcal{F}} 2\pi\delta(\omega - \omega_0)$. So using the modulation property

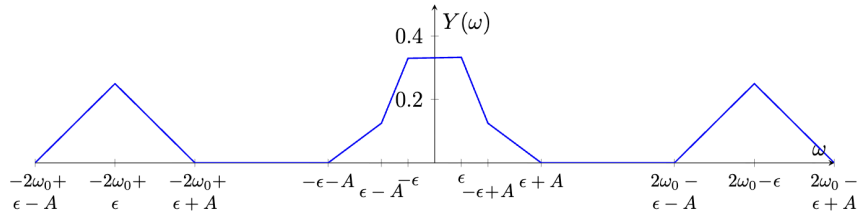
$$\begin{aligned} R(\omega) &= \frac{1}{2\pi} (X * Q_T)(\omega) \\ &= \frac{1}{2\pi} X(\omega) * \left(\frac{2\pi\delta(\omega - \omega_0) + 2\pi\delta(\omega + \omega_0)}{2} \right) \\ &= \frac{1}{2} (X(\omega - \omega_0) + X(\omega + \omega_0)) \end{aligned}$$



For $Y(\omega)$ we have the following

$$\begin{aligned} Y(\omega) &= \frac{1}{2\pi} (R * Q_R)(\omega) \\ &= \frac{1}{2\pi} R(\omega) * \left(\frac{2\pi\delta(\omega - \omega_1) + 2\pi\delta(\omega + \omega_1)}{2} \right) \\ &= \frac{1}{4} (X(\omega - \omega_0) + X(\omega + \omega_0)) * (\delta(\omega - \omega_1) + \delta(\omega + \omega_1)) \\ &= \frac{1}{4} [X(\omega + (2\omega_0 - \epsilon)) + X(\omega + \epsilon) + X(\omega - \epsilon) + X(\omega - (2\omega_0 - \epsilon))] \end{aligned}$$

The additive effect in the middle creates a shape that looks like the Pizza Hut Logo



- (c) [2 pts] Explain why the information-bearing signal x is irrecoverable, even if we send the signal y through an ideal low-pass filter. *Hint:* recall that ϵ is unknown.

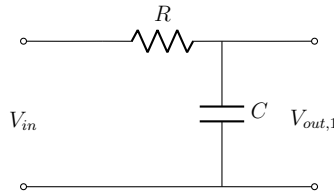
Solution: Looking at the three components of the spectrum $R(\omega)$, and considering that we have no a priori knowledge of the frequency mismatch ϵ , we note that:

- Low pass filtering y would fail to recover x because of the aliasing at the baseband frequencies.
- No further modulation/demodulation of y will work even though at frequencies around $\pm 2\omega_0$ the triangular shape of the original signal x is intact. This is because ϵ is unknown, and hence so are the center frequencies $\pm(2\omega_0 - \epsilon)$

3 Frequency Responses of Low-pass Filters [29 pts]

This question is about different types of low-pass filter and how their frequency responses differ.

- (a) [4 pts] First consider the basic passive, first-order low-pass RC filter:

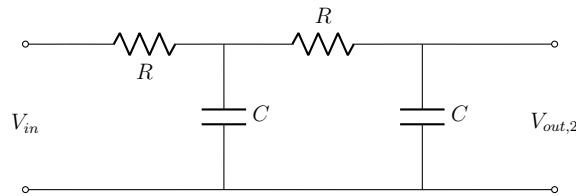


Confirm that you understand why this circuit functions as a low-pass filter by re-deriving the complex transfer function, $V_{out,1}/V_{in}$. Show your derivation.

Solution: Use the voltage divider formula but use impedances

$$\frac{V_{out,1}}{V_{in}} = \frac{\frac{1}{j\omega C}}{R + \frac{1}{j\omega C}} = \frac{1}{1 + j\omega RC}$$

- (b) [6 pts] Now consider what would happen if we connected together two RC stages to make a rudimentary second-order filter:



Derive the complex transfer function $V_{out,2}/V_{in}$ for this circuit, using the voltage-divider principle and complex impedances.

Solution: Let $z_{R1} = R_1 = R$, $z_{R2} = R_2 = R$, $z_{C1} = \frac{1}{C_1 s} = \frac{1}{Cs}$, and $z_{C2} = \frac{1}{C_2 s} = \frac{1}{Cs}$. Then, we can use KVL (2 loops) and KCL to compute the following:

$$\begin{aligned} I &= I_1 + I_2 \\ v_{in} &= z_1 I + z_2 I_2 \\ z_2 I_2 &= (z_3 + z_4) I_1 \\ v_{out} &= z_4 I_1 \end{aligned}$$

Then we can write out the transfer function as:

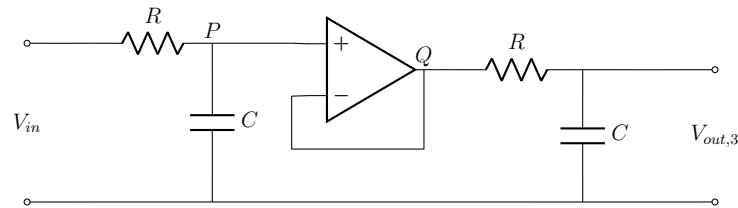
$$\begin{aligned} \frac{v_{out,2}}{v_{in}} &= \frac{z_2 I_1}{z_1 I + z_2 I_2} \\ &= \frac{z_4 I_1}{z_1 (I_1 + I_2) + z_2 I_2} \\ &= \frac{z_4}{z_1 (1 + I_2/I_1) + z_2 I_2/I_1} \\ &= \frac{z_4 z_2}{(z_1 + z_2)(z_3 + z_4) + z_1 z_2} \\ &= \frac{\frac{1}{C^2 s^2}}{R^2 + \frac{2R}{Cs} + \frac{1}{C^2 s^2} + \frac{R}{Cs}} \\ &= \frac{1}{3RCs + R^2 C^2 s^2 + 1} = \frac{1}{3RCj\omega - R^2 C^2 \omega^2 + 1} \end{aligned}$$

This is a fairly fiddly derivation that is helped by remembering that two impedances A and B in parallel have impedance:

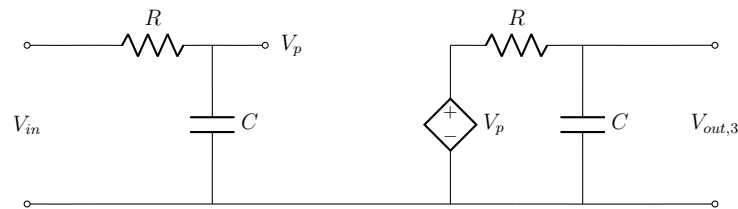
$$A||B = AB/(A + B)$$

Crucially, the transfer function is not simply the product of two first-order transfer functions, as the first stage has finite output impedance and the second stage has finite input impedance. This is precisely why the buffer introduced in the next part of the question has a positive effect on making the transition around the corner frequency sharper (i.e. the response does not roll off as early when the buffer is included).

- (c) [4 pts] The impedances of the two RC stages above interact with each other, which made computing the transfer function a bit more complicated than multiplying together the complex transfer functions for two first-order RC filters. Instead, we can design filters where the different stages are separated from each other by a ‘buffer’, an active circuit whose output voltage equals its input voltage and that has high input impedance and low output impedance. Two RC stages are ‘cascaded’ together as follows:



For simplicity, we can model the voltage follower (unit gain buffer op-amp) as the following circuit



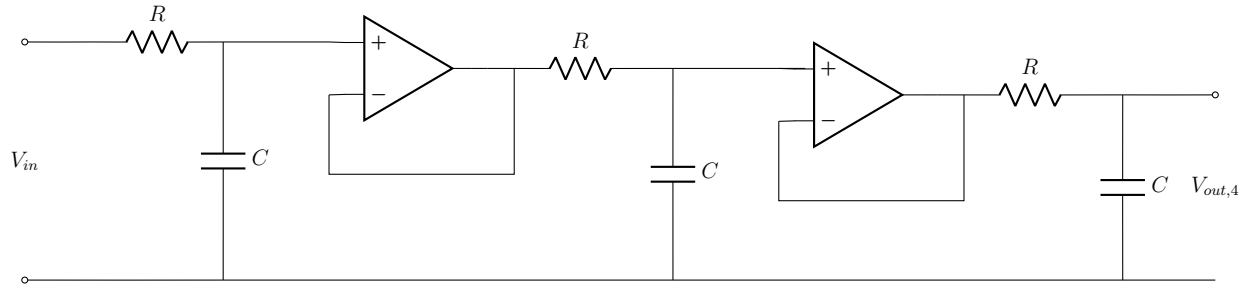
Here, the high input and low output impedances of the buffer mean that the RC stages’ impedances do not interact, and the overall transfer function is the product of the transfer functions of the two RC stages. Thus, write down the complex transfer function $V_{out,3}/V_{in}$ for this circuit.

Solution: Here, the transfer function can be well approximated as:

$$\frac{V_{out,3}}{V_{in}} = \left(\frac{1}{1 + j\omega RC} \right)^2 = \frac{1}{1 + 2j\omega RC - (\omega RC)^2}$$

Writing out the expansion is optional; writing the $\left(\frac{1}{1+j\omega RC} \right)^2$ is enough for full credit.

(d) [3 pt] Similarly, a third-order cascaded RC filter could be constructed as follows:



Write down the complex transfer function $V_{out,4}/V_{in}$ for this circuit.

Solution: Like above, the transfer function can be well approximated as:

$$\frac{V_{out,4}}{V_{in}} = \left(\frac{1}{1 + j\omega RC} \right)^3 = \frac{1}{1 + 3j\omega RC - 3\omega^2 R^2 C^2 - j\omega^3 R^3 C^3} = \frac{1}{1 + 3sRC + 3s^2 R^2 C^2 + s^3 R^3 C^3}$$

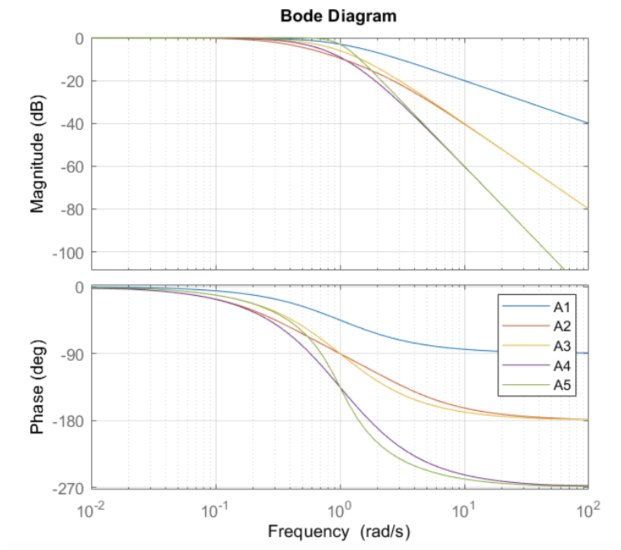
Again, writing out the expansion is optional; writing $\left(\frac{1}{1+j\omega RC} \right)^3$ is enough for full credit.

(e) [7 pts] Create a Bode plot, with magnitude and phase on separate sets of axes, on which you superimpose the frequency responses of each of the four filters considered above. Assume $RC = (1 \text{ second})$ for all filters. Also add the frequency response of the third-order passive Butterworth filter:

$$\frac{V_{out}}{V_{in}} = \frac{1}{s^3 + 2s^2 + 2s + 1}, \quad \text{where } s = j\omega$$

You will need the Control System Toolbox installed to use the `bode()` and `tf()` functions. This toolbox is freely available to registered students.

Solution:



Lines map as follows: A1: first-order RC filter (1.1); A2: second-order simple RC filter (1.2); A3: second-order cascaded filter with buffer between stages (1.3); A4: third-order cascaded filter with buffers between stages (1.4); A5: third-order Butterworth filter.

1 point per correctly plotted line pair (amplitude and phase for a given filter); 1 point for axes being correctly labeled and oriented. Code used is below.

```

1 s = tf('s');
2 A1 = 1/(1+s);
3 A = A1;
4 A3 = A*A;
5 A2 = 1/(1+3*s+s^2);
6 A4 = A*A*A;
7 A5 = 1/(s^3+2*s^2+2*s+1);
8
9 bode(A1)
10 hold on
11 bode(A2)
12 bode(A3)
13 bode(A4)
14 bode(A5)
15 legend
16 grid on

```

- (f) [3 points] Comment on differences between the frequency responses of the rudimentary 2-stage RC filter, found in part (b), and the cascaded filter with buffer, found in part (c). What are some potential pros and cons of the two approaches?

Solution: While both filters have the same roll-off slope at high frequencies, the buffered version has a sharper amplitude–frequency response around the corner frequency. This could be advantageous for maintaining a flat response as close as possible to the corner frequency and would potentially allow the filter to be designed with a lower corner frequency than the passive filter of 1.2, if the filter must pass frequencies up to a given value with minimal attenuation.

On the other hand, the buffered circuit is clearly more complex (still quite simple though) and requires external power to operate the buffer.

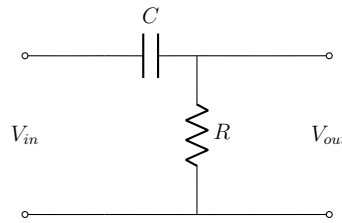
- (g) [2 points] Comment on differences between the frequency responses of the 3-stage cascaded RC filter, found in part (d), and the third-order Butterworth filter. What are some potential pros and cons of the two approaches?

Solution: The third-order cascaded RC filter has a gentler roll-off around the corner frequency than the Butterworth filter. This may have similar disadvantages to those discussed in (f) above: i.e. to pass frequencies up to a given value with minimal attenuation, a higher corner frequency would be needed, which would in turn lead to higher gain at any given frequency well above the corner frequency, and potentially a higher sampling frequency being needed to avoid those parasitic components being aliased.

If the Butterworth filter is implemented with passive components, it would require inductors as well as capacitors and resistors, and the inductors could be rather bulky and lead to unwanted electromagnetic interference with surrounding equipment. On the other hand, Butterworth filters can actually be implemented using active op amps – the same devices used for buffers, but connected so as to mimic the effect of an inductance. So implementation of these two options could be about equally straightforward, or could even be simpler for the Butterworth.

4 Frequency Response of Other Filters [9 pts]

Consider the following filter (note that the resistor and capacitor are switched compared to the low-pass filters in the previous question):



- (a) [3 pts] Derive the complex transfer function, and expressions for the phase and magnitude of the frequency response.

Solution:

$$\frac{v_{out}}{v_{in}} = \frac{R}{R + \frac{1}{j\omega C}} = \frac{j\omega RC}{1 + j\omega RC} \Rightarrow |H| = \frac{\omega RC}{\sqrt{1 + (\omega RC)^2}}$$

The phase response is then

$$\angle \left(\frac{v_{out}}{v_{in}} \right) = \arg \left(\frac{R}{R + \frac{1}{j\omega C}} \right) = \arg(R) - \arg \left(R + \frac{1}{j\omega C} \right) = 0 - \arg \left(R - \frac{1}{j\omega C} \right) = -\text{atan} \left(\frac{-1}{\omega C R} \right) = \text{atan} \left(\frac{1}{\omega RC} \right)$$

- (b) [1 pt] In a few words, describe the behavior of this filter.

Solution: This is a high-pass filter that has a corner frequency of $1/RC$ and a phase lead of 90 degrees at low frequencies that falls to zero at high frequencies.

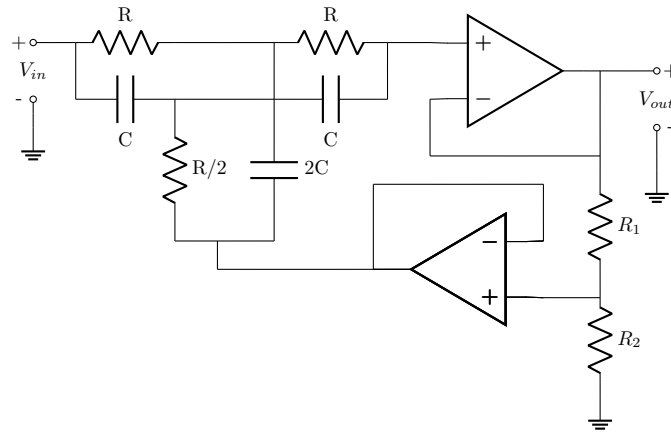
Saying that it is a high-pass filter is enough for the point of credit.

- (c) [1 pt] Discuss any potential purpose/application(s) that you can envisage for a filter of this kind when making measurements.

Solution: This filter could be used if we are trying to sense a high-frequency signal that is superimposed on a lower-frequency signal with larger amplitude, or a high-frequency signal with non-zero average (i.e. non-zero d.c. component). The output of the filter contains the signal of interest but with a zero average. This can be helpful for feeding into a differential amplifier before sampling, since a highly sensitive amplifier can then be used without saturating the output. Alternatively, if the high-pass filtered output is to be fed directly into an ADC, removing a constant offset allows the input range of the ADC to be matched to the amplitude of the signal we are trying to measure, so that resolution is maximized.

Now, consider the following transfer function and circuit for a filter, which is called a 'notch' filter:

$$\frac{s^2 + \omega_0^2}{s^2 + Q^{-1}s + \omega_0^2}, \quad \text{where } s = j\omega$$



- (d) [2 pts] Plot a Bode plot of this filter for a central frequency of $\omega_0 = 120\pi$ rad/s, and a quality factor, $Q = 10$.

Solution:

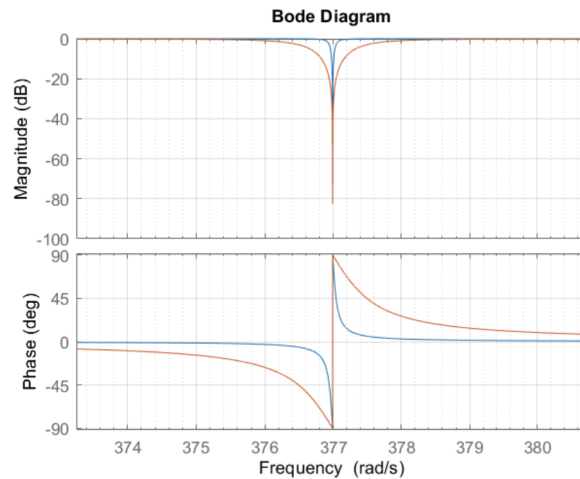


Figure 2: Blue: $Q=10$; Red: $Q=1$

```

1 s = tf('s')
2 omega_0_notch = (2*pi*60);
3 Q = 1e1
4 TFnotch = (s^2+omega_0_notch^2)/(s^2+(1/Q)*s+omega_0_notch^2);
5 figure(10)
6 bode(TFnotch,(2*pi)*logspace(log10(60/1.1),log10(60*1.1),100000))
7 hold on
8 Q = 1
9 TFnotch1 = (s^2+omega_0_notch^2)/(s^2+(1/Q)*s+omega_0_notch^2);
10 figure(10)
11 bode(TFnotch1,(2*pi)*logspace(log10(60/1.1),log10(60*1.1),100000))

```

- (e) [1 pt] What frequency (in Hz) does the filter suppress most strongly? What could be an application of this filter in measurement?

Solution: It suppresses 60 Hz most strongly and could be used to cut ‘hum’ due to electromagnetic interference from power lines.

- (f) [1 pt] What would you expect to happen if you made the quality factor smaller? Try plotting the transfer function for $Q = 1$. Describe what changes and what any implications might be for measurement.

Solution: Reducing the quality factor would broaden the notch in the frequency spectrum so that a wider range of frequencies is suppressed. Depending on how far the frequency of interest lies from the central frequency of the notch, having too low of a Q factor could suppress the signal of interest to some degree in addition to the noise that we are trying to suppress, which would be undesirable. On the other hand, one would not want to have an infinite (or too high) Q factor because that would give a perfectly/excessively sharp notch and there would be no room for error in manufacturing the filter (tolerances on the component values) or in knowing the actual frequency we are trying to suppress.

5 DC Motor Filtering [10 pts]

The speed of a spindle is measured using a small DC motor acting as a generator. The generated voltage is converted into a digital form using an Analogue-to-Digital converter. Commutation in the motor causes a high frequency ripple in the generated voltage, with a peak-to-peak amplitude of 1 V and a frequency which is 100 times that of the rotating spindle.

It is required to provide an **inverting** first order filter that will amplify the generated voltage signal by a factor of 10; it is also required to attenuate the ripple by 60 dB when the spindle speed is 200 Hz.

- (a) [4 pts] Sketch the Bode magnitude and phase shift diagrams of the overall filter circuit by hand, indicating all frequencies and gains of interest.

Solution: Firstly, let's translate our requirements into numbers and equations. The DC gain or low-frequency gain K can be found using the following relation

$$K = 20 \log_{10}(10) = 20 \text{ dB}$$

Thus, the filter must have a voltage gain of 10 at low frequencies or 20 dB. The high-frequency ripple due to commutation must be attenuated by 60 dB when the spindle rotates at 200 Hz. Thus, the ripple frequency

$$f_{\text{ripple}} = 100 \times f_{\text{spindle}} = 100 \times 200 \text{ Hz} = 20 \text{ kHz}$$

Since the DC gain is +20 dB and the required attenuation at 20 kHz is 60 dB, the total drop required is

$$\Delta G = 20 \text{ dB} - (-60 \text{ dB}) = 80 \text{ dB}$$

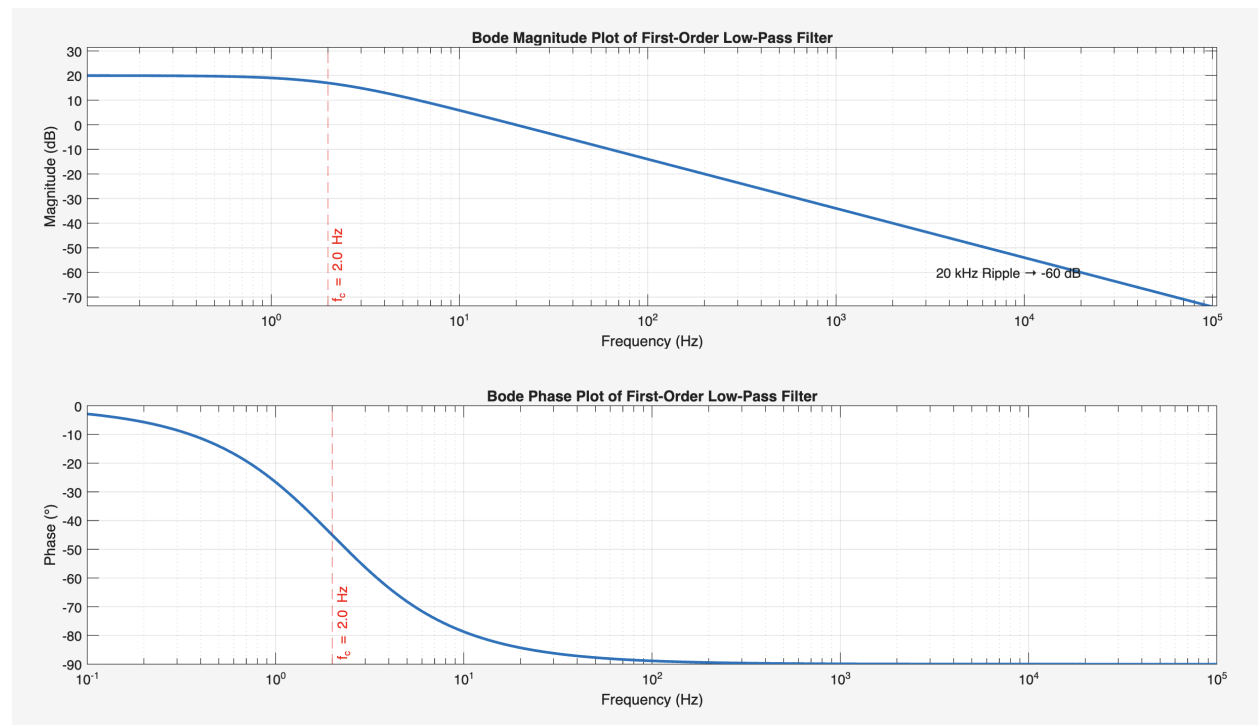
For a first-order low-pass filter, the slope is -20 dB/decade . To achieve an 80 dB reduction, we have

$$\frac{80 \text{ dB}}{20 \text{ dB/decade}} = 4 \text{ decades}$$

Therefore, the cutoff frequency f_c must be **four decades** below the ripple frequency

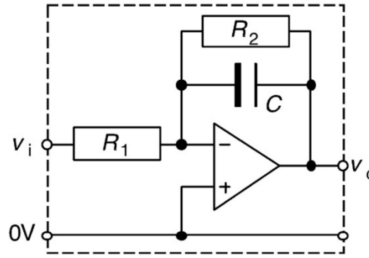
$$f_c = \frac{f_{\text{ripple}}}{10^4} = \frac{20 \text{ kHz}}{10^4} = 2 \text{ Hz}$$

Therefore, the Bode plot will be



- (b) [3 pts] Draw a circuit diagram of the required filter arrangement, including all relevant parameters and circuit elements.

Solution: An inverting first-order active low-pass filter is chosen to provide the desired gain of 10.



- (c) [3 pts] If only $1\mu\text{F}$ capacitors are available, determine values for all resistors in the circuit.

Solution: The passband gain is given by $|H|_{DC}$

$$|H|_{DC} = \left| -\frac{R_2}{R_1} \right| = 10$$

For a single-pole low-pass filter, we have the following cutoff frequency

$$f_c = \frac{1}{2\pi R_2 C} = 2 \text{ Hz} \implies R_2 = \frac{1}{2\pi \times 2 \times 1 \times 10^{-6}} = 80k\Omega$$

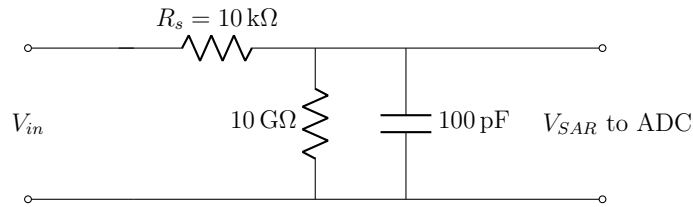
Since $|H|_{DC} = 10$, we have

$$R_1 = \frac{R_2}{10} = 8k\Omega$$

Therefore, the recommended resistor values are approximately: $R_2 = 80k\Omega$, $R_1 = 8k\Omega$, $C = 1\mu\text{F}$. These values set the -3 dB corner frequency near 2 Hz, provide a gain of 10 in the passband, and achieve at 60 dB attenuation by 20 kHz.

6 Settling Error and Time [9 pts]

Consider the following apparatus connected to the input of a successive approximation register (SAR) ADC. The source resistance (inherent in the apparatus generating the signal) is $R_s = 10\text{ k}\Omega$ and the input impedance of the ADC interface is represented by a $10\text{ G}\Omega$ resistor in parallel with a 100 pF capacitor C_{in} .



The voltage seen by the ADC is v_{SAR} . Initially, $v_{SAR} = v_{in} = 0$. Then v_{in} undergoes a step change to $v_{in} = 1\text{ V}$ at time $t = 0$. The circuit will charge up as follows:

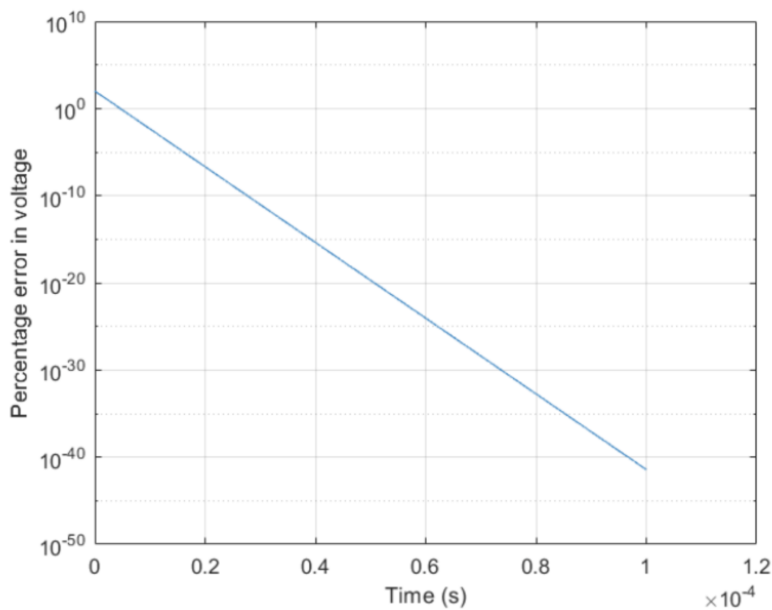
$$v_{SAR}(t) = (1\text{ V}) \left(1 - \exp\left(-\frac{t}{R_s C_{in}}\right) \right).$$

Notes: We ignore the input $10\text{ G}\Omega$ resistance as it is so high, and assume the input impedance of the ADC circuit itself (the circuit that lies to the right of the $10\text{ G}\Omega$ resistor and 100 pF capacitance) is large.

- (a) [2 pts] Plot the percentage error between v_{SAR} and v_{in} for time from 0 to $100\text{ }\mu\text{s}$. Use your judgement to decide whether to use logarithmic, linear, or some other type of scale for the graph.

Solution: Three options are possible. One with a log scale on the vertical (error) axis, a log scale on the time axis, and a linear scale. Any of them is acceptable for 2 points. Our percentage error will be:

$$e = \frac{v_{in} - v_{SAR}(t)}{v_{in}} \cdot 100\% = 1 - \left(\frac{1 - \exp\left(-\frac{t}{R_s C_{in}}\right)}{v_{in}} \right) \cdot 100\%$$



- (b) [1 pt] Calculate how long v_{SAR} takes to fall to an error of 1% after v_{in} changes.

Solution: Start with the percentage error formula from above

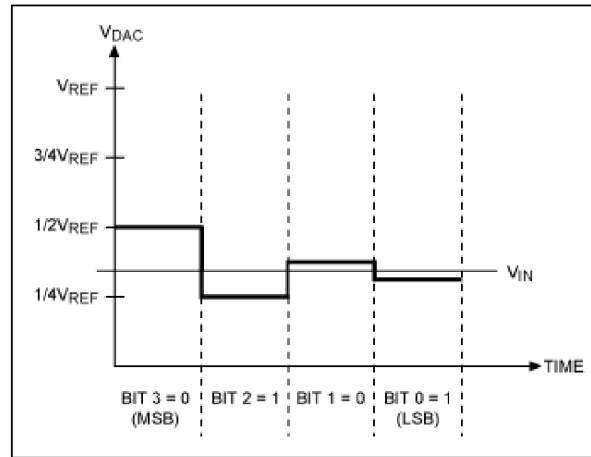
$$e = 1 - \left(\frac{1 - \exp(-\frac{t}{R_s C_{in}})}{v_{in}} \right) = 0.01$$

Then just rearranging and solving for t :

$$\begin{aligned} e &= \exp(-\frac{t}{R_s C_{in}}) \Rightarrow \ln 0.01 = -\frac{t}{R_s C_{in}} \Rightarrow t = -R_s C_{in} \cdot \ln 0.01 \\ \Rightarrow t &= -(10000)(100 \times 10^{-12} \text{F}) \cdot \ln 0.01 \Rightarrow t = 4.6 \mu\text{s} \end{aligned}$$

Therefore, it takes $4.6 \mu\text{s}$ for the voltage v_{SAR} to settle within 1% of the input signal. As the component values are only given to 1 s.f., quoting the answer to 1 s.f. ($5 \mu\text{s}$) is acceptable, but 2 s.f. is ideal.

- (c) [2 pts] The ADC has 16 bits, and the clock frequency of the ADC, f_{clock} , is 10 MHz. In lecture, we described a naïve configuration for an ADC in which the register counted up from 0 and incremented one LSB (least significant bit) at a time, once per clock pulse, until the output of the register matched the input voltage. In a real SAR ADC, the control logic is more sophisticated and faster. The register is initialized in the middle of the measurement range (for an N -bit ADC, at 2^{N-1}). Upon the first clock cycle, the most significant bit of the register is set to the correct value to represent the analog input, and at each subsequent clock cycle the next most significant bit is updated, until the least significant bit is reached and all bits of the register have been updated to represent the input signal as accurately as possible. The digital output matches the analog input. The sampling time of the ADC operated in this way can be approximated as N/f_{clock} . What is the maximum sampling frequency that this ADC can reasonably be operated at?



Solution: Since the ADC has 16 bits, and $f_{\text{clock}} = 10 \text{ MHz}$, we can calculate the sampling time as

$$t_{\text{sampling}} = \frac{N}{f_{\text{clock}}} \Rightarrow f_{\text{sampling}} = \frac{f_{\text{clock}}}{N} = \frac{10 \times 10^6}{16} = 625 \text{ kHz}$$

Therefore, the maximum sampling frequency the ADC can handle, assuming it takes one clock cycle per bit to complete the conversion is roughly 625 kHz

- (d) [2 pts] Taking all these factors into account, what would be a conservative settling time to quote, to guarantee an error below 1% when recording this signal? Explain your reasoning (well-reasoned answers receive complete credit even if they aren't the answer we came up with).

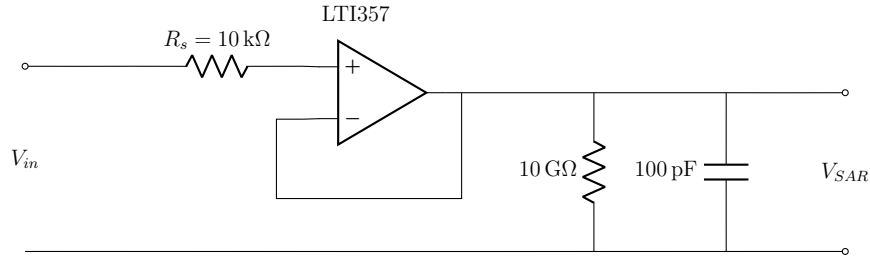
Solution:

A conservative settling time to quote would be the time for the input error to fall to 1% plus two sampling periods. The reason is that we cannot control when exactly the sampling period begins, and

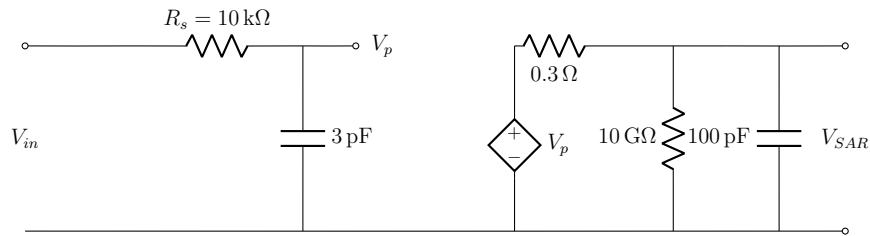
in the worst case, a sampling period could begin just before the input error had fallen to 1%. In that case, the first sample obtained would potentially have an error slightly more than 1%, and to be certain of the ADC recording an error below 1% we should wait for the second sample, which definitely starts to be recorded after the input error is below 1%.

So the settling time would be $4.6\mu s + 2/(625 \text{ kHz}) = 7.8\mu s$. An acceptable answer would also be $4.6\mu s + 1/(625 \text{ kHz}) = 6.2\mu s$ (this allows for one sampling period after the error reaches 1%).

- (e) [1 pt] To try to reduce settling time, a buffer circuit is introduced between the apparatus and the ADC, consisting of an LT1357 operational amplifier configured as a voltage follower. The op-amp functions as a voltage-controlled voltage source that isolates the relatively high source impedance of the apparatus from the ADC's input capacitance. The LT1357 can be assumed to have an input capacitance of 3 pF and an output impedance of 0.3 Ohms, as shown below. For this part of the question, assume the slew rate of the LT1357 is unlimited.



The above buffer circuit can be modeled as



Update your estimate for the time taken for the V_{SAR} to fall to an error of 1%. Explain your reasoning briefly.

Solution: Since we are interested in V_{SAR} , this corresponds to the ADC node. The op-amp's output resistance of 0.3Ω drives the ADC input capacitance of 100pF (the $10 \text{ G}\Omega$ is essentially open on these time scales). Thus, the new time constant is now

$$\tau = RC = (0.3\Omega)(100\text{pF}) = 30\text{ps}$$

The voltage response is then given by

$$v_{SAR}(t) = (1V) \left(1 - \exp\left(-\frac{t}{30\text{ps}}\right) \right)$$

Then, the time for the error to fall to 1% is

$$t_{1\%} = -\ln(0.01) \times 30\text{ps} = 138\text{ps}$$

- (f) [1 pt] Now consider that the specified slew rate of the LT1357 is in fact $600 \text{ V}/\mu s$. (The circuit model for the LT1357 shown above is approximate, so there are some other internal imperfections that determine the slew rate.) Is slew rate a significant limiter for the overall response speed of the measurement system? How does considering the slew rate affect your estimate of the settling time of the overall measurement, if at all? What if you had used a basic LM741 op amp, with a slew rate of $0.5 \text{ V}/\mu s$?

Solution: At the maximum slew rate of $600V/\mu s$, the time taken for the output to rise from 0 to 1 V (the asymptotic voltage value in this sensing circuit) would be 1.6 ns or 1600 ps, which is about 10 times larger than the RC delay alone, but still much shorter than the sampling time of $1/(625 \text{ kHz}) = 1.6\mu s$. Therefore the overall sampling time with the buffer would be close to $21.6\mu s = 3.2\mu s$, with the RC delay being negligible.

If using an LM741, the time taken to slew from 0 to 1 V output is $2\mu s$, which is comparable to the sampling time, and the slew rate capability of the op amp is now significant in determining the settling time, which can be approximated as $2\mu s + 2/(625 \text{ kHz}) = 5.2\mu s$. Again, with all of these, answers given to 1 s.f. are acceptable, as are answers assuming that only one sampling period is needed after the input error settles to below 1%.